

Aura-AI

An Image Generation Platform

**A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF
REQUIREMENT**

FOR THE AWARD OF THE DEGREE

MASTER OF COMPUTER APPLICATIONS (MCA)

OF

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

BY

Reebaik Hamilton

Reg No : 24PMC142



**MARIAN COLLEGE
KUTTIKKANAM**
(AUTONOMOUS)

MAKING COMPLETE

Marian College Kuttikkanam Autonomous

Peermade, Kerala – 685 531

2025

Aura-AI

An Image Generation Platform

**A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILMENT OF REQUIREMENT
FOR THE AWARD OF THE DEGREE
MASTER OF COMPUTER APPLICATIONS
(MCA)
OF
MAHATMA GANDHI UNIVERSITY, KOTTAYAM
BY**

REEBAIK HAMILTON

Reg No : 24PMC142



**MARIAN COLLEGE
KUTTIKKANAM**

(AUTONOMOUS)

MAKING COMPLETE

Marian College Kuttikkanam Autonomous

Peermade, Kerala – 685 531

2025

A Project Report on

Aura-AI

An Image Generation Platform

**SUBMITTED IN PARTIAL FULFILMENT OF REQUIREMENT
FOR THE AWARD OF THE DEGREE**

**MASTER OF COMPUTER APPLICATIONS
(MCA)**

**OF
MAHATMA GANDHI UNIVERSITY, KOTTAYAM**

**By
REEBAIK HAMILTON
Reg No : 24PMC142**

Under the guidance of
SR.ITALIA JOSEPH MARIA
Assistant Professor
PG Department of Computer Applications
Marian College Kuttikkanam Autonomous



**MARIAN COLLEGE
KUTTIKKANAM**

(AUTONOMOUS)

MAKING COMPLETE

Marian College Kuttikkanam Autonomous

Peermade, Kerala – 685 531

2025

PG DEPARTMENT OF COMPUTER APPLICATIONS

Marian College Kuttikkanam Autonomous

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

KUTTIKKANAM – 685 531, KERALA.

CERTIFICATE

This is to certify that the project work entitled

Aura AI

is a bonafide record of work done by

Reebaik Hamilton

Reg. No. 24PMC142

In partial fulfillment of the requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS [MCA]

During the academic year 2025-2026

Sr.Italia Joseph Maria

Assistant Professor

PG Department of Computer Applications

Marian College Kuttikkanam Autonomous

Mr Win Mathew John

Head of the Department

PG Department of Computer Applications

Marian College Kuttikkanam Autonomous

External Examiner

ACKNOWLEDGEMENT

I am deeply grateful to the Almighty for giving me the strength, wisdom, and opportunity to successfully complete this project. It is with immense pleasure that I present this report, and I owe my sincere gratitude to several individuals who have been instrumental throughout this journey.

I would like to extend my heartfelt gratitude to **Mr. Win Mathew John**, Head of the Department (MCA), for his constant support and guidance, which has been invaluable in the successful completion of this project.

I am profoundly thankful to my project guide, **Sr. Italia Joseph Maria**, Assistant Professor at the PG Department of Computer Applications, Marian College Kuttikkanam, for her insightful guidance, encouragement, and unwavering support during every stage of the project.

I also wish to thank all the faculty members of the **PG Department of Computer Applications** for their assistance, encouragement, and constructive feedback, which greatly contributed to the successful completion of my project.

Finally, I would like to express my deepest gratitude to my friends, family, and everyone who provided their support and encouragement throughout this journey. Your kindness and belief in my abilities have made this accomplishment possible.

Thank you all for being a part of this enriching and fulfilling journey.

Reebaik Hamilton

ABSTRACT

Aura AI is an innovative AI-powered image generation platform designed to empower creativity through cutting-edge technology. It serves as a digital canvas where users can craft visually stunning images using text prompts, powered by advanced models like Stable Diffusion. The platform fosters a vibrant creative community by enabling users to generate, like, share, and manage their generated artwork through a personalized gallery.

With a strong focus on accessibility and control, Aura AI implements a **subscription-based model** that ensures high-quality service while supporting platform sustainability. Only subscribed users can access image generation features, ensuring efficient resource allocation and a premium experience. The project is built using the MERN stack (MongoDB, Express, React, Node.js), integrates **Cloudinary** for image storage, and is optimized for responsiveness and scalability.

Key features include user authentication, image gallery management, download/share capabilities, admin controls, and a secure subscription system. Future enhancements aim to integrate real-time AI customization, advanced prompt tuning, and mobile app support, making Aura AI a creative powerhouse in the evolving landscape of AI-generated art.

Chapter	TABLE OF CONTENTS	Page No
1	Introduction	1
	1.1 Problem Statements	2
	1.2 Proposed System	2
	1.3 Features of the Proposed System	2
2	Functional Requirements	4
3	Non-functional Requirements	7
4	UML Diagrams	10
	4.1 Use Cases	11
	4.2 Use Case Diagram	18
	4.3 Activity Diagram	19
	4.4 Class Diagram	21
5	Test Cases	22
	5.1 Login Page	23
	5.2 Registration Page	25
	5.3 Home Page	26

5.4	Image Generation Page	27
5.5	User Gallary Page	28
6	Input Design and Output Design	29
7	System Implementation	36
8	Future Enhancements	46
9	Conclusion	48
Annexure		
A	Screenshots	

TABLE INDEX

Page No

7.4.1	User Table	39
7.4.2	Image Table	39
7.4.3	User gallery	40
7.4.4	Admin table	40
7.4.5	Password Resest Token Table	41
7.4.6	Subscription Table	41

FIGURE INDEX

Page No

4.2	User Case Diagram	18
4.3.1	Activity Diagram-User	19
4.3.2	Activity Diagram-Admin	20
4.4	Class Diagram	21

1. INTRODUCTION

1.1 Problem Statement

In an era where AI-driven creativity is rapidly advancing, there is a lack of accessible and community-oriented platforms that allow users to easily generate, manage, and share AI-generated images. Most existing tools are either overly technical or lack community features. **Aura AI** aims to fill this gap by offering a streamlined, subscription-based platform that enables users to unleash their creativity through AI-generated art while engaging with a like-minded community.

1.2 Proposed System

Aura AI provides a full-featured, user-friendly web application for generating images using advanced AI models like Stable Diffusion. The platform is designed around a **subscription model**, allowing only subscribed users to access image generation features. With options to like, share, and manage personal galleries, Aura AI promotes creativity, collaboration, and accessibility while ensuring system performance and resource efficiency.

1.3 Features of the Proposed System

1. User Registration and Login:

Secure authentication system to ensure protected and personalized access.

2. Subscription-Based Access:

Only subscribed users can generate images, promoting efficient resource usage and service quality.

3. Image Generation from Prompts:

Users input text prompts to generate high-quality AI images using Stable Diffusion.

4. Gallery Management:

Users can save, view, and manage their generated images in a personal gallery.

5. Like and Share Functionality:

Users can like images and share them across platforms, enhancing visibility and engagement.

6 Admin Dashboard:

Admins can manage users, monitor content, and handle inappropriate images or activity.

2. FUNCTIONAL REQUIREMENTS

1. User authentication and registration:

- Users will be able to register with a username, email, and password.
- Users can securely log in and log out of the platform.
- A password recovery mechanism will be available for account recovery.

2. Subscription Management

- Users must purchase a subscription to access image generation features.
- Subscription status is verified before allowing image creation.

3. Image Generation and Gallery Management

- Subscribed users can generate images using text prompts.
- Users can save generated images to their personal gallery.
- Users can delete images from their gallery at any time.

4. Like And Share Functionality :

- Users can like or remove likes on generated images.
- Users can share images via social platforms or copy shareable links.
- Like counts are visible to all users on each image.

5. User profile management:

- A list of generated and saved images is accessible from their profile.

6. Search Functionality

- Users can search for images by prompt keywords or by creator name.

7. Admin Fuctionality

- Admins can delete inappropriate images, or user accounts.
- Admins can edit user details like username and email
- Admins can assign or revoke admin privileges to other users.

3. NON-FUNCTIONAL REQUIREMENTS

1. *Performance:*

- The platform efficiently handles multiple concurrent user sessions, including image generation requests, likes, and gallery operations.
- Image generation and page load times are optimized to complete within two to three seconds on average.
- Database queries for retrieving user images, likes, and subscriptions are optimized to maintain responsiveness.

2. *Scalability:*

- Aura AI is built to scale, supporting an increasing number of users, generated images, and real-time interactions.
- The infrastructure accommodates future upgrades such as multi-model support, AI customization, and premium-tier features.

3. *Security:*

- Passwords are securely hashed using modern algorithms like bcrypt to prevent unauthorized access.
- JWT (JSON Web Token) is used for secure, stateless user authentication and authorization across protected routes.

3. *Accessibility:*

- Aura AI is accessible 24/7, ensuring continuous availability for global users and system administrators.

4. *Usability:*

- The platform is designed to be intuitive, with a clean interface for prompt input and gallery navigation.

- Forms are simple, intuitive, and display clear error messages to guide users through the process of correcting mistakes

6. Interoperability:

- The platform functions seamlessly on popular web browsers, including Chrome, Firefox, Safari, and Edge.
- Cross-platform compatibility is ensured, enabling users to access and interact with the platform efficiently from different operating systems and devices.

4. UML DIAGRAMS

4.1 Use Cases

Use Case 1: User Registration

Use Case ID: UC-001

Actors: Visitor

Description: This use case describes how a new visitor registers for an account on the Aura AI platform.

Preconditions: The visitor has access to the registration page.

Postconditions: A new user account is created, and the user can log in.

Main Flow:

Visitor navigates to the registration page.

Visitor enters required information (username, email, password).

System validates the input and creates a new user account.

User receives a confirmation message and is redirected to the login page.

Alternative Flow:

If the input is invalid (e.g., email already in use, password too weak), an error message is shown.

Use Case 2: User Login

Use Case ID: UC-002

Actors: Registered User

Description: This use case describes how a user logs into their Aura AI account.

Preconditions: The user has a valid account.

Postconditions: The user is authenticated and redirected to the home page.

Main Flow:

User navigates to the login page.

User enters valid credentials (email and password).

System verifies credentials and returns a JWT.

User is redirected to the home page.

Alternative Flow:

If credentials are incorrect, an error message is displayed.

Use Case 3: Generate AI Image

Use Case ID: UC-003

Actors: Logged-in User (with active subscription)

Description: This use case describes how a user generates an image using AI.

Preconditions: The user is logged in and subscribed.

Postconditions: A new image is generated and displayed to the user.

Main Flow:

User navigates to the image generation page.

User enters a prompt.

System sends the prompt to the backend (Stable Diffusion API).

Image is generated and displayed.

Alternative Flow:

If the user is not subscribed, access is denied with a message.

If the prompt is invalid or empty, an error message is shown.

Use Case 4: Save Image to Gallery

Use Case ID: UC-004

Actors: Logged-in User (Subscribed)

Description: This use case describes how a user saves a generated image to their personal gallery.

Preconditions: User is logged in and subscribed.

Postconditions: Image URL is stored in the user's gallery.

Main Flow:

User clicks the "Save to Gallery" button.

Image is uploaded to Cloudinary.

Image URL is saved to MongoDB under the user's record.

Confirmation message is displayed.

Use Case 5: Like an Image

Use Case ID: UC-005

Actors: Logged-in User

Description: This use case describes how a user likes an image.

Preconditions: The user is logged in.

Postconditions: The like count of the image is updated.

Main Flow:

User views an image on the home or detail page.

User clicks the "Like" button.

System checks if the user already liked the image.

If not, like count is incremented and stored in the database.

Updated like count is displayed.

Alternative Flow:

If the user already liked the image, clicking “Like” again will remove the like and decrement the count.

Use Case 6: Admin Deletes User or Image

Use Case ID: UC-006

Actors: Admin

Description: This use case describes how an admin deletes an inappropriate user or image.

Preconditions: Admin is logged in.

Postconditions: The selected user or image is deleted from the database and/or Cloudinary.

Main Flow:

Admin navigates to the Admin Dashboard.

Admin selects a user or image to delete.

Admin clicks the “Delete” button.

Confirmation is shown, and on confirmation, the item is deleted.

Alternative Flow:

If the admin cancels, no action is taken.

Use Case 7: Subscribe to a Paid Plan

Use Case ID: UC-007

Actors: Logged-in User

Description: This use case describes how a user subscribes to a paid plan to unlock image generation and saving features.

Preconditions: The user is logged in.

Postconditions: The user's account is upgraded with a valid subscription.

Main Flow:

User navigates to the subscription page.

User selects a subscription plan.

User enters payment details and confirms.

System processes the payment via a payment gateway.

Upon success, subscription status is updated in the user's profile.

User receives a confirmation message and can now generate and save images.

Alternative Flow:

If the payment fails, an error message is shown and subscription is not activated.

Use Case 8: Share or Download an Image

Use Case ID: UC-008

Actors: Logged-in User or Visitor

Description: This use case describes how a user or visitor shares or downloads an image from the platform.

Preconditions: The image must exist and be visible to the user.

Postconditions: The image is either downloaded to the user's device or a shareable link is copied.

Main Flow (Download):

User or visitor clicks the "Download" button on an image.

System prepares the image for download.

Image is downloaded to the user's device.

Main Flow (Share):

User or visitor clicks the “Share” button.

A shareable link or social media options are displayed.

User copies the link or shares directly.

Alternative Flow:

If the image URL is invalid or inaccessible, an error message is shown.

Use Case 9: Delete an Image from User Gallery

Use Case ID: UC-009

Actors: Logged-in User

Description: This use case describes how a user deletes a previously saved image from their personal gallery.

Preconditions: The user must be logged in and own the image.

Postconditions: The image is removed from the database and Cloudinary.

Main Flow:

User navigates to their gallery.

User selects an image to delete.

User clicks the “Delete” button.

System prompts for confirmation.

Upon confirmation, image is deleted from Cloudinary and MongoDB.

User sees a success message.

Alternative Flow:

If the user cancels the confirmation, the image remains unchanged.

If the deletion fails, an error message is shown.

4.2 Use Case Diagram

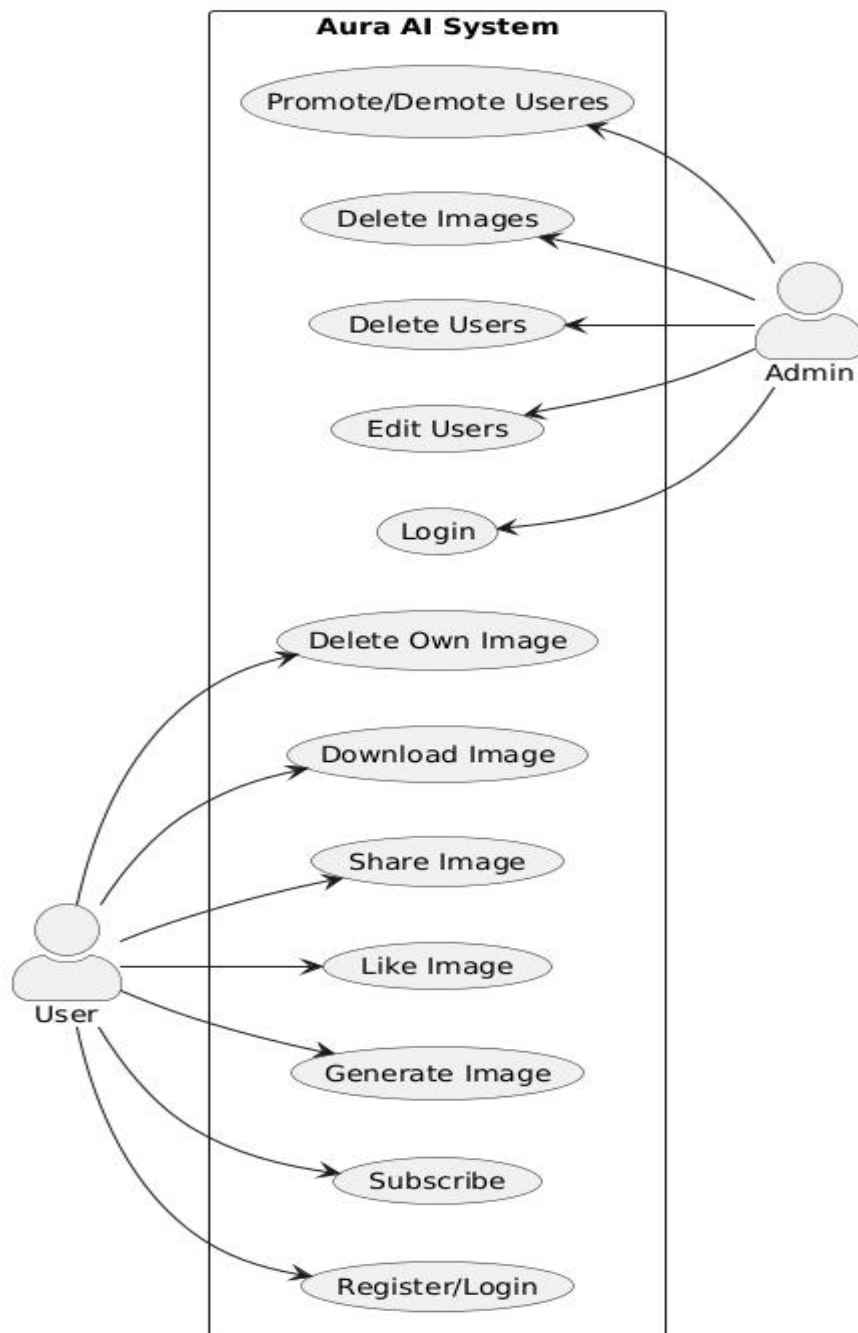


Fig 4.2 Use case Diagram

4.3 Activity Diagram

4.3.1 User Activities:

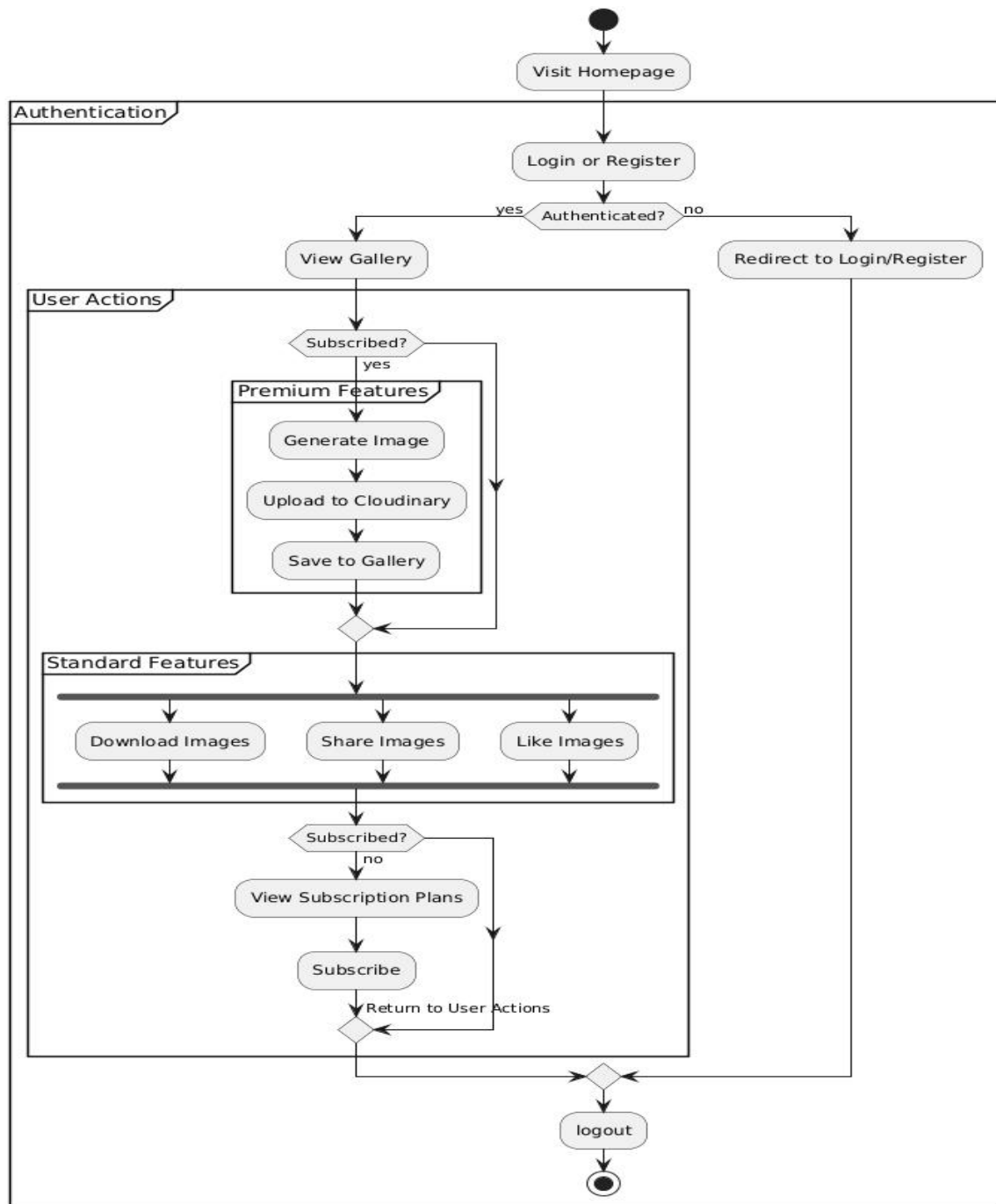


Fig 4.3.1 User Activity Diagram

4.3.2 Admin Activities:

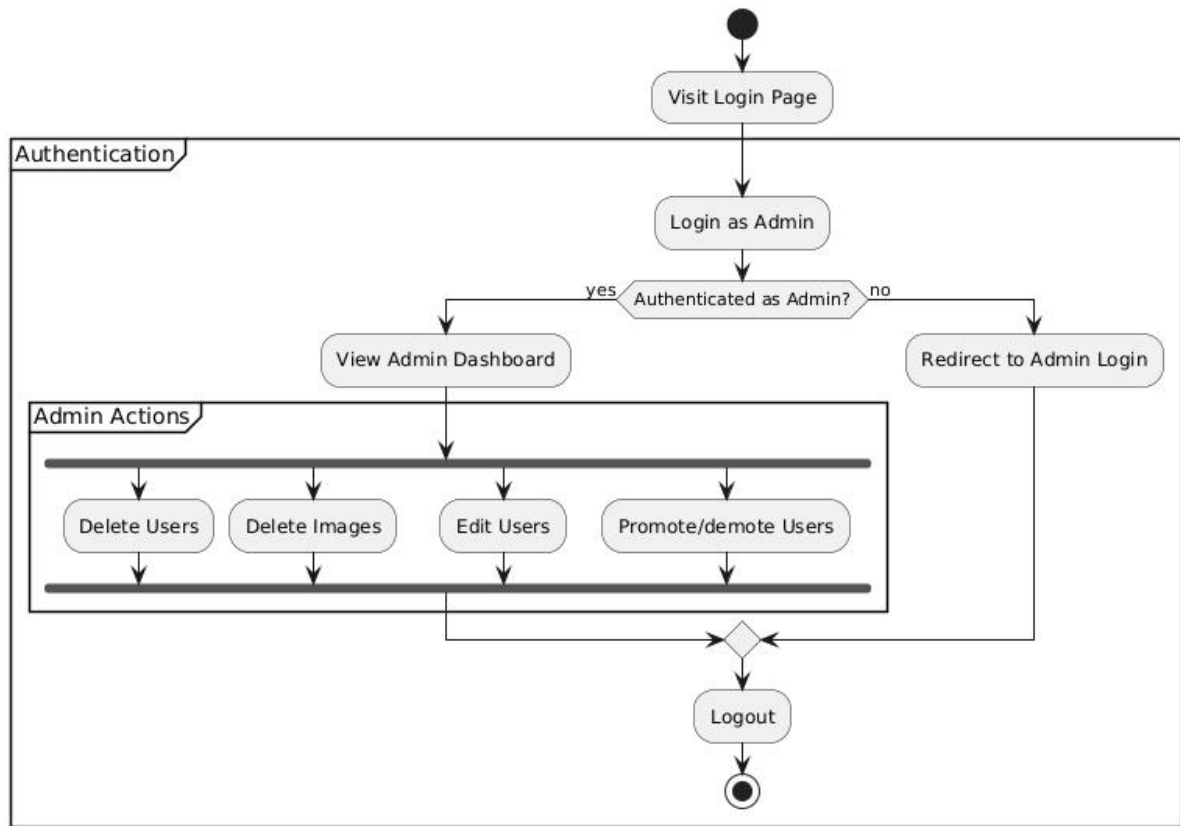


Fig 4.3.2 Admin Activity

4.4 Class Diagram

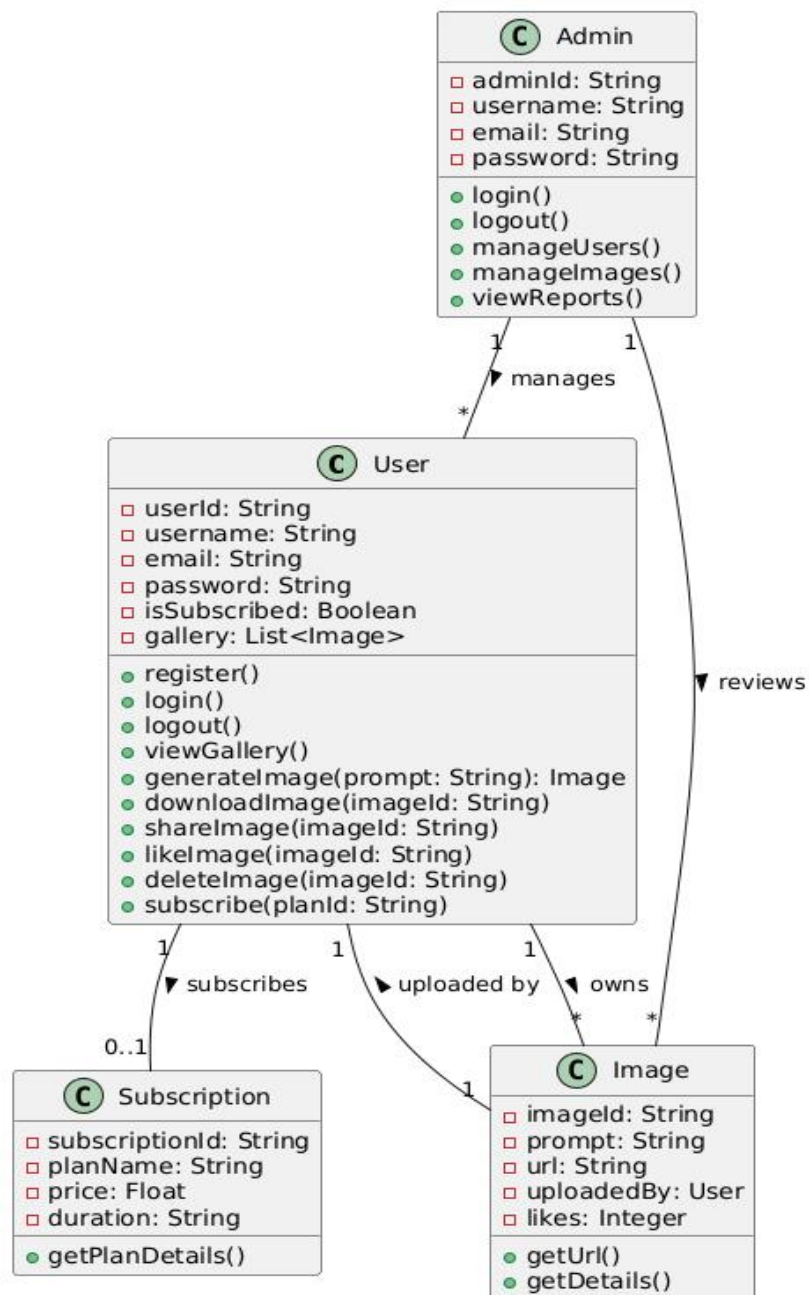


Fig 4.4 Class Diagram

5. TESTING

5.1 LOGIN PAGE

Test Cases	Feature	Description	Steps To Execute	Expected Results
Valid Login	Login	Test with valid credentials	1. Navigate to the Login Page. 2. Enter valid username. 3. Enter valid password. 4. Click on "Login" button.	User should be successfully logged into their account.
Invalid Email	Login	Test with invalid Email	1. Navigate to the Login Page. 2. Enter invalid Email 3. Enter valid password. 4. Click on "Login" button.	System should display an error message for an invalid Email

Invalid Password	Login	Test with invalid password	1. Navigate to the Login Page. 2. Enter valid username. 3. Enter invalid password. 4. Click on "Login" button.	System should display an error message for an invalid password.
Empty Fields	Login	Test with empty fields	1. Navigate to the Login Page. 2. Leave username and password fields empty. 3. Click on "Login" button.	System should prompt the user to fill in both username and password fields.

5.2 REGISTRATION PAGE

Test Cases	Feature	Description	Steps To Execute	Expected Results
Valid Registration	Registration	Test with valid user information	1. Navigate to the Registration Page. 2. Enter valid user details (username, email, password). 3. Click on "Register" button.	User should be successfully registered and directed to the login page or a success message should be displayed.
Existing Email	Registration	Test with an existing email	1. Navigate to the Registration Page. 2. Enter an email that is already registered. 3. Fill other details. 4. Click on "Register" button.	System should prompt the user that the email is already in use.
Weak Password	Registration	Test with a weak password	1. Navigate to the Registration Page. 2. Enter a weak password (less than minimum requirements). 3. Fill other details. 4. Click on "Register" button.	System should display an error message indicating password requirements are not met.

Empty Fields	Registration	Test with empty fields	1. Navigate to the Registration Page. 2. Leave one or more fields empty. 3. Click on "Register" button.	System should prompt the user to fill in all required fields.
--------------	--------------	------------------------	---	---

5.3 HOME PAGE

Test Case	Feature	Description	Steps To Execute	Expected Result
Guest View	Public Feed	Access as not logged in user	Open Home Page without logging in	Should display gallery with "Login" button
Logged-In View	Authenticated Feed	Access after login	Login → Navigate to Home	Should show images + 'Generate' button
Like Post	Like Function	Like an image	Login → Click 'Like' under a post	Like count increases by 1
Unlike Post	Like Function	Unlike a previously liked image	Click Like again on same image	Like count decreases by 1
Like Limit	Validation	Prevent double-like by same user	Click Like repeatedly on same image	System only counts one like

5.4 IMAGE GENERATOR PAGE

Test Case	Feature	Description	Steps To Execute	Expected Result
Generate Image	AI Image Generation	Enter prompt & generate image	Login → Navigate to Generate Page → Enter valid prompt → Click 'Generate'	AI should return image and display with options
Empty Prompt	Prompt Validation	Try generating with no input	Leave prompt empty → Click 'Generate'	Error: 'Prompt is required'
Save Image	Save to Gallery	Save generated image to personal gallery	After image is generated → Click 'Save'	Image saved to user gallery
Non-Subscribed User	Access Control	Prevent non-paid user from generating	Login with free account → Go to Generate Page → Try generating image	Show subscription required message

5.5 USER GALLERY PAGE

Test Case	Feature	Description	Steps To Execute	Expected Result
View Gallery	User Image History	Check saved images in profile	Login → Navigate to Profile	Gallery displays saved/generated images
Like Visibility	Feedback Reflection	Check like count is visible	Open any saved image	Shows like count

6. INPUT DESIGN AND OUTPUT DESIGN

Introduction to Input Design:

The input design in the AI Image Generator platform is crafted to provide a smooth and secure user experience for different roles (Guests, Registered Users, Admins). Each form and input field is validated on both client and server sides to ensure robust functionality, enhance user experience, and safeguard against invalid data entry.

Forms and Input Fields

HTML5 validation and regex patterns for Home Page (Guest/Registered)

Search Form

Fields:

Search Bar (Text Field): Allows searching for image prompts or creators.

Validation: Optional; must be a valid string (no special characters).

Registration Form (For New Users)

Fields:

Username (Text Field): Unique identifier for the user.

Validation: Required; minimum 5 characters.

Email (Email Field): Used for authentication and communication.

Validation: Valid email format required.

Password (Password Field): User's login credential.

Validation: Minimum 8 characters, must include letters and numbers.

Confirm Password (Password Field): Confirms password match.

Validation: Must match the "Password" field.

Login Form

Fields:

Email (Email Field): User's registered email.

Validation: Must be a valid email format.

Password (Password Field): Account password.

Validation: Required and must match stored credentials.

Image Generator Page (For Subscribed Users Only)

Fields:

Prompt (Text Area): The input text for image generation.

Validation: The field must not be empty

Generate Button: Triggers the image generation process.

Subscription Check: Users must have an active subscription.

Validation: Server-side check for subscription status.

Edit Profile Form

Fields:

Username (Text Field): Update current username.

Validation: Required; must not already exist.

Email (Email Field): Update email address.

Validation: Valid email format and uniqueness check.

Change Password Form

Fields:

Old Password (Password Field): Current password.

Validation: Required.

New Password (Password Field): New password.

Validation: Minimum 8 characters, must be alphanumeric.

Confirm New Password: Confirms new password.

Validation: Must match new password.

Subscription Form

Fields:

Payment Details (Card Info, Name, etc.): Collected via secure gateway (e.g., Stripe).

Validation: Handled by payment processor.

Subscribe Button: Initiates the subscription process.

Validation and Error Handling

Client-Side Validation

immediate feedback.

Example:

```
pattern="[A-Za-z0-9]{8,}" for passwords.
```

Server-Side Validation

Mongoose model validation and Express middleware checks.

Email format and uniqueness.

Subscription validation before allowing image generation.

Prompt length and content.

Error Messages

Examples:

“Password must contain at least 8 characters and include both letters and numbers.”

“Email already in use.”

“You must be subscribed to generate images.”

6.2 Output Design

Introduction to Output Design:

Output design delivers data back to users in a meaningful and organized way. The goal is to ensure clarity and accessibility in how images, profile info, and system notifications are presented across the application.

User Interface Elements

Home Page

Featured Gallery: Displays top-liked/generated images.

Search Results: Lists image thumbnails with prompt, author, like count, and share/download buttons.

User Profile Page

Displays:

User's generated images (gallery view).

Image Detail Page

Displays:

Full image with prompt.

Creator's name.

Like button with count.

Share and download options.

Admin Dashboard

Displays:

All Users: View and manage user details, roles, and status.

All Images: View, delete inappropriate content.

Error and Success Messages

Success Messages:

“Image generated and saved successfully.”

“Subscription activated.”

“Password changed successfully.”

Error Messages:

“Invalid credentials.”

“Only subscribed users can access this feature.”

“Please enter a valid prompt.”

“Failed to upload image. Try again.”

7. SYSTEM IMPLEMENTATION

7.1 Introduction

The AI Image Generation Website is developed using the **MERN stack** (MongoDB, Express.js, React, and Node.js) for full-stack JavaScript development. The frontend is designed using **React with Vite**, styled with **Tailwind CSS**, and enhanced by **Redux** for state management. The backend leverages **Express.js** with **MongoDB** as the primary database. **Cloudinary** is used for image storage and retrieval, and image generation is powered by a locally hosted **Stable Diffusion model** integrated into the backend. Only **subscribed users** are allowed to generate and save images.

7.2 Project Structure

ai-image-generator/

```
|
|
| — client/           # Frontend (React with Vite)
|   |
|   | — src/
|   |   |
|   |   | — components/    # Reusable components (Navbar, ImageCard, etc.)
|   |   | — pages/         # Page components (Home, Login, Register, Profile, Generator)
|   |   | — redux/         # Redux store, slices
|   |   | — App.jsx        # Main component
|   | — server/           # Backend (Node.js + Express)
|   |   |
|   |   | — controllers/    # Logic for auth, image, and admin ope
|   |   | — models/        # Mongoose schemas (User, Image)
|   |   | — middleware/    # Auth, role checks
|   |   | — routes/        # Express route handlers
|   |   | — utils/         # Cloudinary and Stable Diffusion utils
```

```
| └─ server.js      # Entry point
|
| └─ .env           # Environment variables
| └─ README.md
```

7.3 System Components

Frontend (React + Vite):

Pages: Register, Login, Home, Generator, Profile, Edit Profile, Image Detail, Admin Dashboard.

Redux: Manages authentication, user state, and image data.

Tailwind CSS: Ensures a responsive and modern UI.

JavaScript: Handles interactions like likes, downloads, and image preview modals.

Backend (Node.js + Express):

Routing: RESTful APIs for users, image handling, and admin actions.

Authentication: JWT-based login and middleware-protected routes.

Role Management: Admins can manage users and delete images.

Subscription Logic: Paid users are granted access to image generation.

Cloudinary:

Stores generated images with metadata (prompt, user, URL, timestamp).

Stable Diffusion:

Receives prompts via backend API and returns generated image links.

7.4 Database Design and Models :

7.4.1. Users - Stores user details

Field Name	Data Type	Constraints
id	UUID	PRIMARY KEY, UNIQUE, NOT NULL
username	VARCHAR(255)	UNIQUE, NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	TEXT	NOT NULL
isAdmin	BOOLEAN	DEFAULT FALSE, NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

7.4.2. Images - Stores generated images and metadata

Field Name	Data Type	Constraints
id	UUID	PRIMARY KEY, UNIQUE, NOT NULL
user_id	UUID	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE, NOT NULL
image_url	TEXT	NOT NULL
prompt	TEXT	NOT NULL
likes	INT	DEFAULT 0
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

7.4.3. User_gallery - Links users to their saved images

Field Name	Data Type	Constraints
id	UUID	PRIMARY KEY, UNIQUE, NOT NULL
user_id	UUID	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE, NOT NULL
image_id	UUID	FOREIGN KEY REFERENCES images(id) ON DELETE CASCADE, NOT NULL
saved_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

7.4.4. Admin - Stores admin credentials and access rights

Field Name	Data Type	Constraints
id	UUID	PRIMARY KEY, UNIQUE, NOT NULL
user_id	UUID	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE, UNIQUE, NOT NULL
role	VARCHAR(50)	DEFAULT 'admin', NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

7.4.5. Password_resets - Stores password reset tokens

Field Name	Data Type	Constraints
id	UUID	PRIMARY KEY, UNIQUE, NOT NULL
user_id	UUID	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE, NOT NULL
token	VARCHAR(255)	UNIQUE, NOT NULL
expires_at	TIMESTAMP	NOT NULL

7.4.6. Subscriptions - Stores user subscription status

Field Name	Data Type	Constraints
id	UUID	PRIMARY KEY, UNIQUE, NOT NULL
user_id	UUID	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE, NOT NULL
status	VARCHAR(50)	DEFAULT 'free', NOT NULL
started_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
expires_at	TIMESTAMP	NOT NULL

7.5 URL Routing and API Endpoints

Frontend Routes:

/ – Home page with gallery

/login – User login

/register – User registration

/generate – Image generator (for subscribed users)

/profile/:userId – View profile

/edit-profile – Update username/email/profile pic

/image/:imageId – Image detail view

/admin – Admin dashboard (only visible to admins)

Backend API Routes:

POST /api/auth/register

POST /api/auth/login

POST /api/images/generate

POST /api/images/upload

GET /api/images/user/:id

GET /api/images/all

PATCH /api/users/update-profile

DELETE /api/images/delete/:id

POST /api/images/like/:id

GET /api/users/all (*Admin*)

DELETE /api/users/:id (*Admin*)

7.6 Frontend Templates & Integration

Base Layout: Shared elements (navbar, footer) rendered via App.jsx.

Tailwind Utility Classes: Used across pages for layout, color, spacing, and responsiveness.

Dynamic Components: Modal for image previews, conditional rendering for buttons (e.g., “Like”, “Download” only for logged-in users).

Redux Store: Maintains global auth and gallery state.

7.7 Authentication & Authorization

JWT Authentication: Token-based login with middleware to protect routes.

Roles:

Admin: Full access, including image moderation and user management.

User: Access to profile, gallery, image generation (if subscribed).

Password Hashing: Implemented via bcrypt during registration.

Session Persistence: Tokens stored in local storage for auto-login.

7.8 Forms and Validation

Frontend:

React-controlled forms with client-side validation (email, password, required fields).

Backend:

Validates unique email, strong password, and proper prompt format.

Error handling and messages via centralized middleware.

7.9 Core Business Logic

Image Generation:

Only subscribed users can generate images.

Prompt is sent to Stable Diffusion → image is returned → uploaded to Cloudinary → saved to MongoDB.

Gallery:

Homepage fetches all images.

User profile shows personal gallery.

Likes:

Users can like/unlike images. Like count is stored in the image model.

Profile:

Users can update their username, email (with availability check), bio, and profile picture.

Admin Dashboard:

View and delete users, moderate images

7.10 Testing and Debugging

Postman:

Used for testing all backend routes (auth, image generation, CRUD).

Manual UI Testing:

Each frontend interaction (image gen, like, login, logout, profile update) tested manually.

Console Logging & Redux DevTools:

Used to trace frontend actions and backend responses.

Error Boundaries:

React error boundaries show graceful fallback UI.

Cloudinary Logs:

Monitored for image upload success/failure.

Stable Diffusion API Logs:

Debug prompt handling and image return time.

8. FUTURE ENHANCEMENTS

Advanced Admin Dashboard:

The Admin Dashboard can be enhanced with comprehensive analytics on user activity, image generation trends, and storage usage. Metrics such as the most active users, most liked/shared/downloaded images, and subscription insights can help admins make informed decisions. Bulk user and image management options can streamline administrative tasks.

AI Prompt Enhancement & Auto-Tagging:

Integrating advanced Natural Language Processing (NLP) can help improve user prompts by offering autocomplete suggestions, grammar checks, or even AI-driven enhancements for more effective image generation. Generated images can also be auto-tagged using image recognition for better organization and search.

Image Recommendation System:

An intelligent recommendation engine can be developed to suggest similar or trending images to users based on their previous likes, generated images, or interactions. This will improve engagement by helping users discover visually similar or popular content.

In-App Image Editing Tools:

Incorporating lightweight image editing features (crop, filter, adjust colors) post-generation will provide users with more control over the final look of their images without needing third-party tools.

Public Profiles and User Portfolios:

Users can be given the option to create public profiles that showcase their generated image galleries. This can promote creativity and help users build an audience or community around their art.

Mobile Application:

A mobile version of the app for Android and iOS will offer users greater flexibility to generate, download, and manage their AI-generated images. Features like push notifications, offline access to the gallery, and mobile-exclusive tools can enhance the user experience.

.

9. CONCLUSION

Aura AI Image Generator platform reimagines the process of creative visual generation by combining the power of AI with a seamless and user-friendly interface. Designed with scalability, performance, and user experience in mind, the platform enables users to create, save, and share unique AI-generated images with ease.

Built using the MERN stack and integrated with modern tools like Cloudinary for image hosting, the platform ensures reliable performance and scalable media handling. Subscription-based access control helps manage resources effectively while offering premium features to paying users.

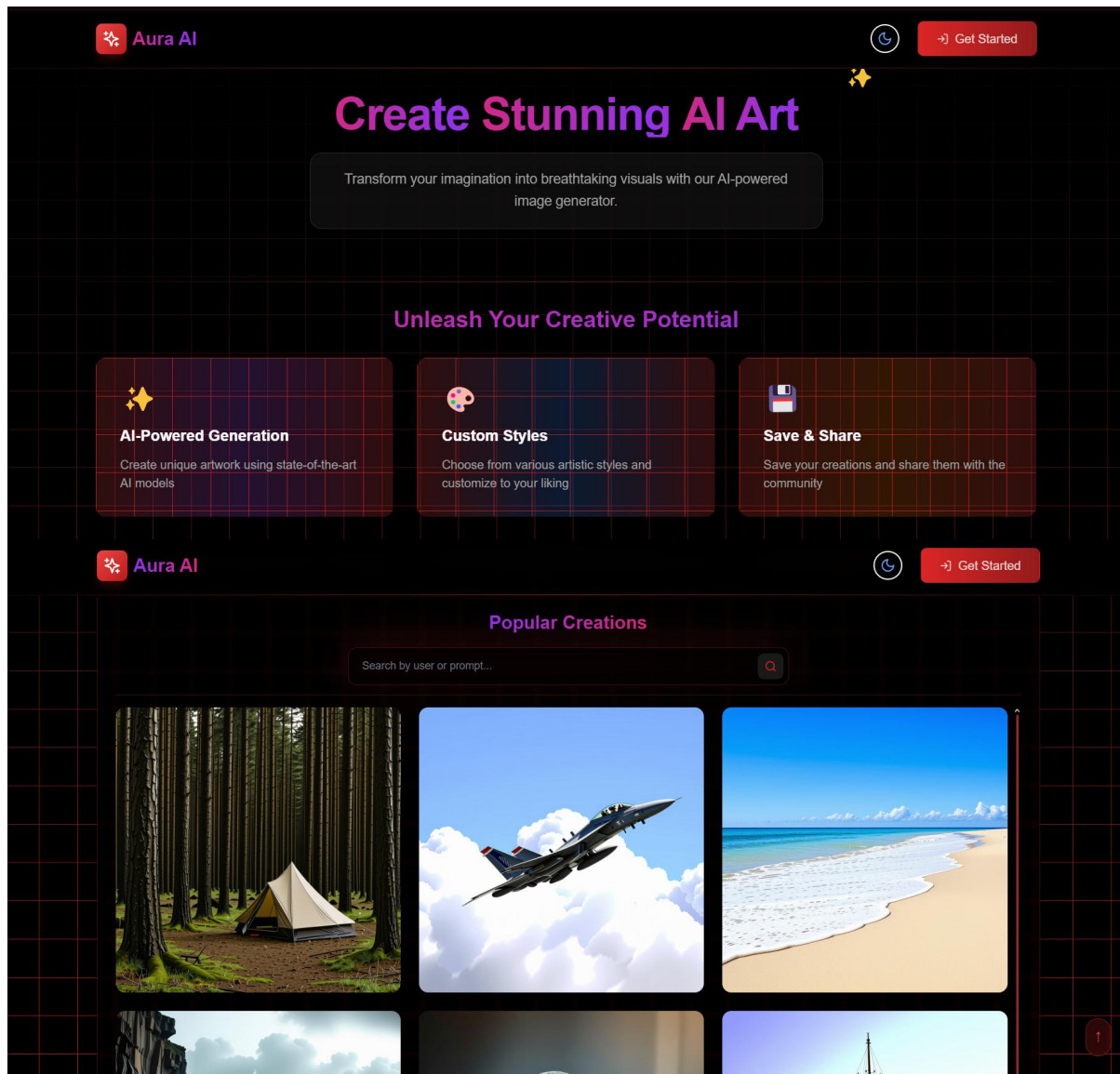
From secure authentication and image management to community features like likes, comments, and sharing, the platform provides a well-rounded experience for both casual users and serious creatives. Admins are equipped with essential tools to monitor platform activity and enforce content guidelines, ensuring a safe and vibrant creative space.

The platform is more than a utility—it's a community-driven hub for AI-assisted creativity. As it evolves, features like recommendation engines, mobile apps, and AI-driven suggestions will further enhance its capabilities. With a commitment to innovation, security, and usability, the AI Image Generator is poised to become a leading platform for generating and exploring AI-powered visuals.

10. ANNEXURE

10.1 SCREENSHOTS

10.1.1 Home page



10.1.2 Register page

Create Account

Username

Email

Password

- At least 8 characters
- Contains a number
- Contains a special character
- Contains a capital letter

Confirm Password

Register

Or

Sign up with Google

10.1.3 Login page

Welcome Back

Username

Password

☐ Remember me [Forgot Password?](#)

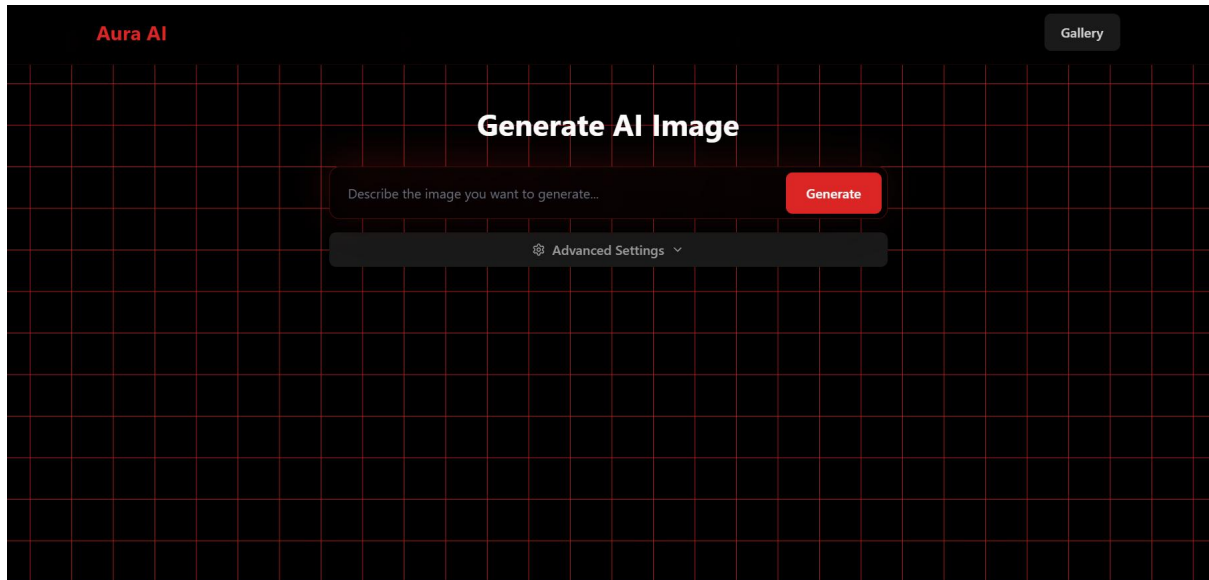
Login

Or

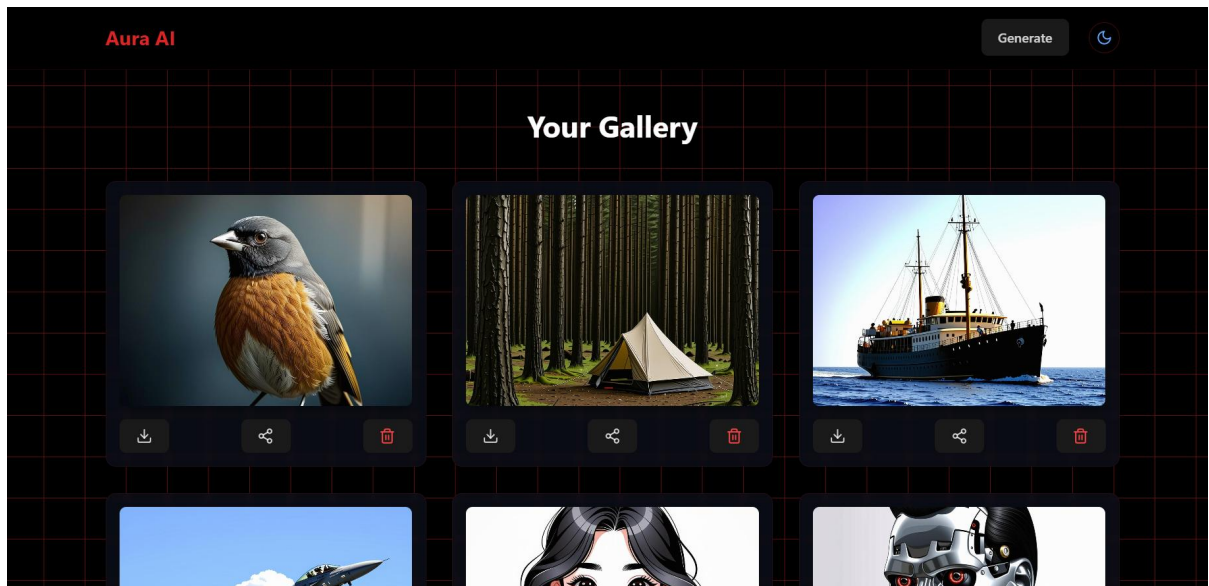
Sign in with Google

Not yet registered? [Register now](#)

10.1.4 Image Generation page



10.1.5 User gallery



10.1.6 Subscription page

Aura AI

Choose Your Plan

Select the perfect plan for your creative needs

Basic Plan	Pro Plan Popular	Premium Plan
₹499 /month	₹999 /month	₹1999 /month
☒ 50 Image Generations	☒ 150 Image Generations	☒ 400 Image Generations
☒ Standard Resolution	☒ HD Resolution	☒ 4K Resolution
☒ Basic Support	☒ Priority Support	☒ 24/7 Premium Support
☒ Valid for 30 days	☒ Advanced Settings Control	☒ Custom Model Fine-tuning
☒ Valid for 30 days	☒ Valid for 30 days	☒ Bulk Generation
☒ Valid for 30 days	☒ Valid for 30 days	☒ Valid for 30 days
Subscribe Now	Subscribe Now	Subscribe Now

10.1.7 Image Deatil Page

[← Back](#)



 **test1**
12 days ago

Prompt
a guy with a long coat standing on a cliff a ocean waves clash against the cliff side its really stormy and windy and make it look like a painting

Generation Parameters

 STEPS
50

 GUIDANCE
0.5

 DIMENSIONS
1024 x 1024

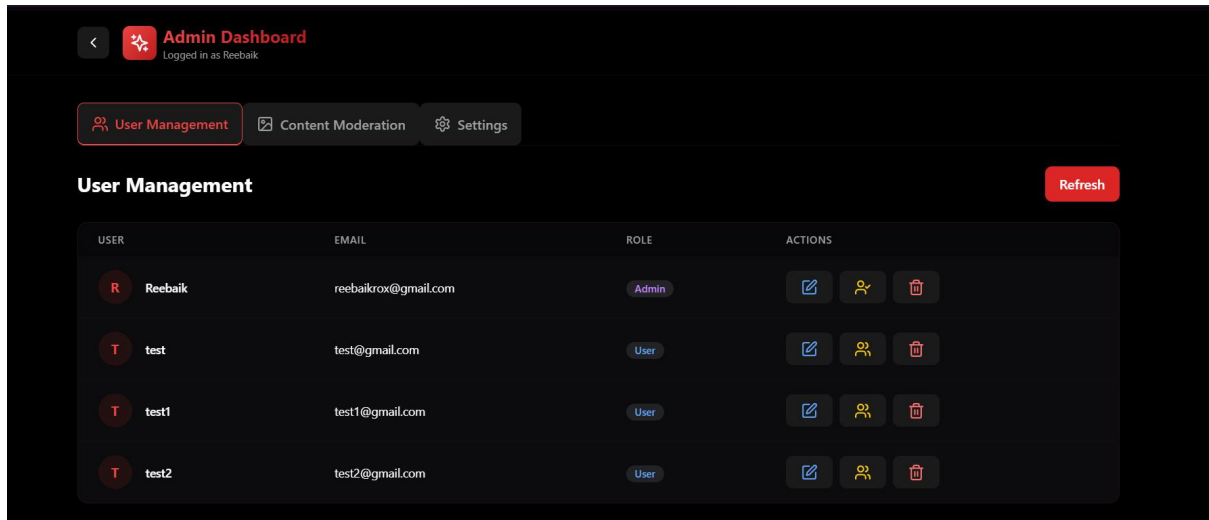
Want to create your own amazing AI-generated images?
[Sign In to Create](#)

 0 Likes

 Download

 Share

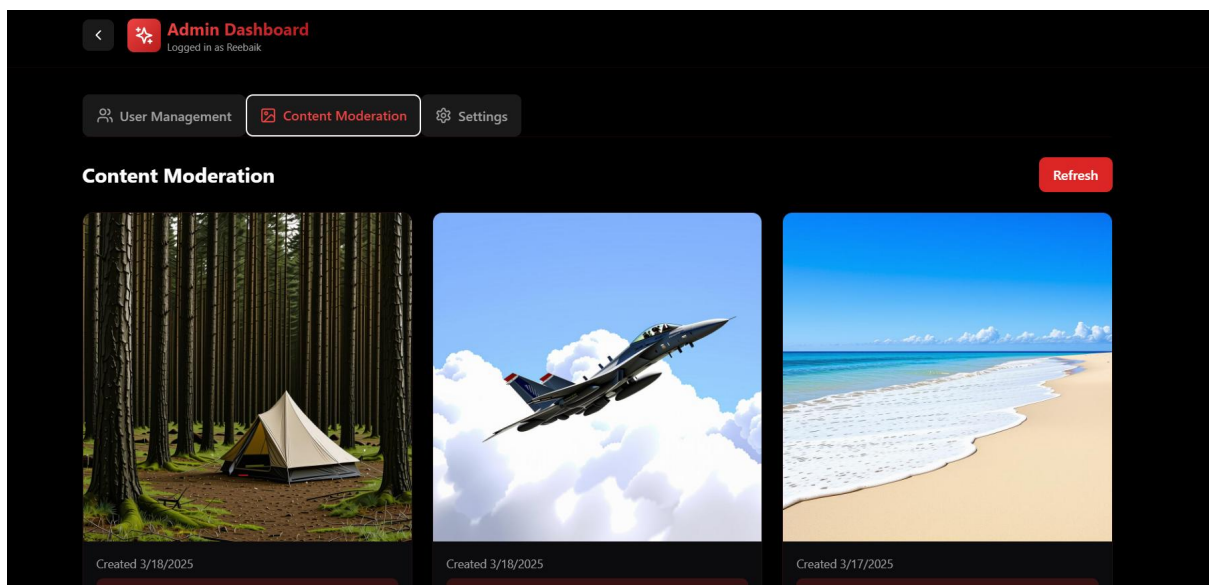
10.1.9 Admin dashboard Manage users Page



The screenshot shows the 'Admin Dashboard' interface. At the top, there's a header with a back arrow, a user icon, and the text 'Admin Dashboard' and 'Logged in as Reebaik'. Below the header, there are three tabs: 'User Management' (selected), 'Content Moderation', and 'Settings'. The main section is titled 'User Management' and features a 'Refresh' button. It contains a table with the following data:

USER	EMAIL	ROLE	ACTIONS
 Reebaik	reebaikrox@gmail.com		  
 test	test@gmail.com		  
 test1	test1@gmail.com		  
 test2	test2@gmail.com		  

10.1.10 Admin dashboard Content Moderation



The screenshot shows the 'Admin Dashboard' interface. At the top, there's a header with a back arrow, a user icon, and the text 'Admin Dashboard' and 'Logged in as Reebaik'. Below the header, there are three tabs: 'User Management', 'Content Moderation' (selected), and 'Settings'. The main section is titled 'Content Moderation' and features a 'Refresh' button. It displays three content items in a grid:

- 
Created 3/18/2025
- 
Created 3/18/2025
- 
Created 3/17/2025

